BTK AKADEMİ

İLERİ SEVİYE PYTHON PROGRAMLAMA DİLİ

CSV/JSON Dosyaları

EBRU AYDOĞAN

CSV Nedir?

- CSV **(Comma Separated Values)** dosyaları, verilerin virgüllerle (bazen noktalı virgül veya tab ile) birbirinden ayrılarak tutulduğu düz metin dosyalarıdır.
- Excel, Google Sheets veya herhangi bir veri tabanı ile uyumlu çalışır.
- **Neden Kullanırız?**
  - Basittir ve çok az yer kaplar.
  - Veri transferi için evrensel bir standarttır.
  - Python ile veriyi işlemek (okumak/yazmak) son derece hızlıdır.
- **Yapı:**
  - İlk satır genellikle sütun başlıklarını (urun_adi, fiyat) barındırır.
  - Takip eden satırlar ise bu başlıkların karşılığı olan verileri içerir.

CSV Dosyası Okuma (csv.reader)

- Python'da csv modülünü kullanarak dosyalar üzerinde işlem yaparız.
- Dosyayı açarken open() fonksiyonu içinde mutlaka encoding='utf-8' parametresini kullanmalıyız (Türkçe karakterler için).
- csv.reader() metodu, dosyayı okuyarak her satırı bir **liste (list)** yapısına dönüştürür.
- **Dikkat:** reader nesnesi bir iterator'dır, yani üzerinde for döngüsü ile tek tek dolaşabiliriz.
- **Kod Mantığı:**
  - Dosyayı "read" (r) modunda aç.
  - csv.reader objesi oluştur.
  - for döngüsü ile satırları ekrana yazdır.

BTK AKADEMİ

Örnek Uygulama (csv.reader)

```python
import csv

# urunler.csv dosyasını açıyoruz
with open('urunler.csv', 'r', encoding='utf-8') as file:
    # csv.reader objesini oluşturuyoruz
    reader = csv.reader(file)

    # Her bir satır bir listeye karşılık gelir
    for satir in reader:
        # Satırları ekrana yazdırıyoruz
        # satir[0] -> urun_adi, satir[1] -> fiyat
        print(f"Ürün: {satir[0]} - Fiyat: {satir[1]}")

# Not: İlk satırda başlıklar olduğu için onları
# atlamak isterseniz 'next(reader)' kullanabilirsiniz.
```

BTK AKADEMİ

DictReader ile Sözlük Tabanlı Okuma

- reader kullanırken veriye satir[0], satir[1] gibi indislerle erişiriz. Ancak bu tehlikelidir; sütun sırası değişirse veya dosya yapısı güncellenirse kodumuz çalışmaz veya yanlış veriyi okur.
- DictReader ise CSV'nin **ilk satırını (başlıkları) otomatik olarak anahtar (key)** olarak belirler.
- Böylece her bir satırı bir liste değil, **Dictionary (Sözlük)** yapısında okuruz. satir['urun_adi'] gibi doğrudan sütun ismiyle veriye erişmek, kodun okunabilirliğini mükemmel seviyeye çıkarır.

BTK AKADEMİ

Örnek Uygulama (DictReader)

```python
import csv

# urunler.csv dosyasını DictReader ile açıyoruz
with open('urunler.csv', 'r', encoding='utf-8') as file:
    # DictReader, ilk satırı otomatik olarak header (sözlük anahtarı) yapar
    reader = csv.DictReader(file)

    # Her bir satır artık bir dictionary (sözlük) objesidir
    for satir in reader:
        # İndislerle değil, sütun isimleriyle veriye ulaşıyoruz
        print(f"Ürün: {satir['urun_adi']} - Fiyat: {satir['fiyat']}")
```

BTK AKADEMİ

Neden DictReader Kullanmalıyız?

- **Okunabilirlik:** satir[0] mı daha anlaşılır, satir['urun_adi'] mi?
- **Dinamik Yapı:** CSV dosyasına yeni bir sütun eklense bile, kullandığınız diğer sütunların yerini değiştirmedikçe kodunuz hatasız çalışmaya devam eder.
- **Hata Payı:** İndis numaralarıyla (0, 1, 2) uğraşmadığınız için hata yapma olasılığınız düşer.
- **Entegrasyon:** Okuduğumuz veri doğrudan bir dictionary olduğu için, bu veriyi başka bir fonksiyonumuza veya bir Product nesnemize parametre olarak göndermek çok daha kolaydır.

CSV Veri Yazma (csv.writer)

- csv.writer nesnesi, Python'daki **listeleri** alıp CSV formatına çevirerek dosyaya yazar.
- Veriyi satır satır (writerow) veya toplu liste halinde (writerows) ekleyebilirsiniz.
- **Dikkat:** newline='' parametresini kullanmazsanız, Windows işletim sisteminde satırlar arasında istenmeyen boş satırlar oluşur.

Örnek Uygulama (csv.writer)

```python
import csv

# 'w' modu: Dosyayı yazma modunda açar (dosya yoksa oluşturur).
# newline='': CSV yazarken satırlar arası boşluk oluşmasını engeller.
with open('urunler.csv', 'w', encoding='utf-8', newline='') as file:
    # CSV writer nesnesini oluşturuyoruz
    writer = csv.writer(file)

    # 1. Başlık satırını ekleyelim (Listeyi doğrudan CSV'ye yazar)
    writer.writerow(['urun_adi', 'fiyat'])

    # 2. Tek satır ekleme (writerow)
    writer.writerow(['Laptop', '25000'])

    # 3. Birden fazla satırı toplu ekleme (writerows)
    # İç içe listelerden oluşan bir yapı gönderiyoruz
    urunler = [['Mouse', '500'], ['Klavye', '1200'], ['Monitor', '4000']]
    writer.writerows(urunler)

print("Dosya başarıyla oluşturuldu.")
```

BTK AKADEMİ

DictWriter ile Veri Yazma

- writer kullanırken verilerin sırasını (indisleri) ezbere bilmeniz gerekir. Eğer ['Laptop', '25000'] listesinde yanlışlıkla ['25000', 'Laptop'] derseniz, veri bozulur.
- DictWriter, **Sözlük (Dictionary)** yapısıyla çalışır. Sütun isimlerini (fieldnames) en başta tanımlarsınız. Böylece veri sırası karışsa bile DictWriter doğru sütuna doğru veriyi yerleştirir.
- writeheader() metodu, tanımladığınız sütun başlıklarını CSV dosyasının en üstüne otomatik yazar.

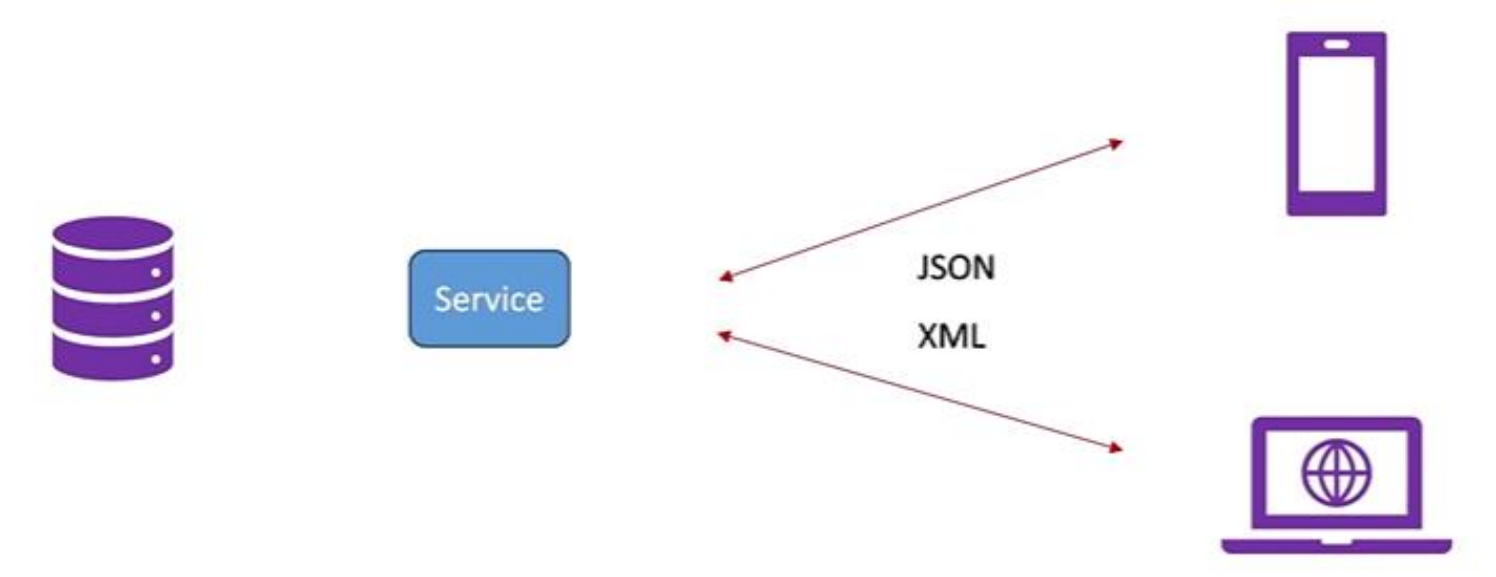
Örnek Uygulama (DictWriter)

```python
import csv

# CSV'de hangi sütunların olacağını belirliyoruz
fieldnames = ['urun_adi', 'fiyat']

# Yazılacak veriler (Liste içinde sözlükler)
urunler = [
    {'urun_adi': 'Laptop', 'fiyat': '25000'},
    {'urun_adi': 'Mouse', 'fiyat': '500'},
    {'urun_adi': 'Klavye', 'fiyat': '1200'}
]

with open('urunler_dict.csv', 'w', encoding='utf-8', newline='') as file:
    # DictWriter nesnesini oluştururken sütun isimlerini (fieldnames) veriyoruz
    writer = csv.DictWriter(file, fieldnames=fieldnames)

    # Başlıkları (fieldnames'teki değerleri) dosyaya yazar
    writer.writeheader()

    # Tüm sözlük listesini tek seferde yazıyoruz
    writer.writerows(urunler)

    # Dilerseniz tek bir sözlük de ekleyebilirsiniz
    writer.writerow({'urun_adi': 'Monitor', 'fiyat': '4000'})

print("DictWriter ile dosya başarıyla oluşturuldu.")
```

BTK AKADEMİ

JSON Nedir?

- JSON (JavaScript Object Notation), veri depolamak ve farklı sistemler arasında veri transferi yapmak için kullanılan hafif (lightweight) bir formatıdır.



JSON Nedir?

- Neden İnternetin Dilidir?
  - Bir sunucu (Service), veritabanından aldığı bilgiyi farklı platformlara (Mobil cihazlar, Web tarayıcıları) göndermek istediğinde ortak bir dile ihtiyaç duyar. JSON, tüm diller (Python, JS, Java vb.) tarafından kolayca anlaşıldığı için bu köprü görevini görür.
- Avantajları:
  - İnsanlar tarafından okunması ve yazılması çok kolaydır.
  - Programlar tarafından analiz edilmesi (parse) çok hızlıdır.
  - Hiyerarşik yapıya izin verir (Bir verinin içinde başka veriler, listeler saklanabilir).

JSON Veri Yapısı ve Görünümü

- JSON yapısı Python'daki **Dictionary (Sözlük)** ve **List (Liste)** yapısına neredeyse tıpatıp benzer.
- **Temel Kurallar:**
  - Veriler **Key-Value (Anahtar-Değer)** çiftleri şeklinde tutulur.
  - Anahtarlar her zaman çift tırnak (" ") içinde olmalıdır.
  - Veriler virgülle ayrılır.
  - Süslü parantezler { } objeleri, köşeli parantezler [ ] dizileri (listeleri) temsil eder.

JSON Veri Yapısı ve Görünümü

- Örnek Veri

```
{
    "id": 1,
    "productName": "IPhone 15",
    "price": 50000,
    "isPublished": true,
    "tags": ["cep telefonu", "elektronik"],
    "category": {
        "id": 1,
        "categoryName": "Telefon"
    }
}
```

BTK AKADEMİ

JSON Veri Yapısı ve Görünümü

- JSON vs Python Veri Tipleri Karşılaştırması
  - JSON'dan Python'a veri geçtiğinde tipler otomatik olarak şu şekilde eşleşir:

JSON	**Python**
Object { }	Dictionary dict
Array [ ]	List list
String " "	String str
Number (int/real)	int / float
true / false	True / False
null	None

BTK AKADEMİ

JSON Dosyası Okuma (Deserializing)

- **Deserialization**, JSON formatındaki bir veriyi (metin/string veya dosya) Python'un anlayabileceği veri tiplerine (dict, list vb.) dönüştürme işlemidir.
- Python'da bu iş için iki temel fonksiyon kullanılır:
  - **json.loads()**: Bir JSON **string** (metin) verisini Python objesine çevirir. (Sondaki **'s'** harfi string'den gelir).
  - **json.load()**: Bir JSON **dosyasını** okuyarak Python objesine çevirir.

Örnek Uygulama (json.load)

- Bilgisayarımızda kayıtlı olan bir .json dosyasını sisteme dahil etmek için load fonksiyonunu with open yapısıyla beraber kullanırız.

```python
import json

# product.json dosyasını okuma modunda açıyoruz
with open("product.json", "r", encoding="utf-8") as f:
    # Dosya içeriğini doğrudan Python sözlüğüne çeviriyoruz
    data = json.load(f)

# Artık 'data' değişkeni bir sözlük (dict) gibi davranır
print(data["productName"]) # IPhone 15
print(data["price"])       # 50000
# İçiçe yapıya erişim
print(data["category"]["categoryName"]) # Telefon
```

BTK AKADE

Örnek Uygulama (json.loads)

- Bazen veri bize bir dosyadan değil, bir internet sitesinden (API) doğrudan metin olarak gelir. Bu durumda loads kullanırız.

```python
import json

# JSON formatında bir string (metin) değişkeni
# Dikkat: JSON içinde çift tırnak olduğu için dışarıda tek tırnak kullandık.
person_string = '{"name":"Ali","languages":["python","c#"]}'

# String veriyi Python Dictionary yapısına çeviriyoruz
result = json.loads(person_string)

print(type(result)) # <class 'dict'>
print(result["name"]) # Ali
print(result["languages"][0]) # python
```

BTK AKADEMİ

JSON Veri Yazma (Serializing)

- **Serialization**, Python'daki bir sözlük (dict) veya liste (list) yapısını, kalıcı olarak saklanabilecek veya başka bir sisteme gönderilebilecek bir JSON formatına dönüştürme işlemidir.
- Python'da bu iş için iki temel fonksiyon kullanılır:
  - **json.dumps()**: Python objesini JSON formatında bir **string** (metin) değişkenine çevirir.
  - **json.dump()**: Python objesini doğrudan bir **dosyaya** JSON formatında yazar.

BTK AKADEMİ

Örnek Uygulama (json.dumps)

- Veriyi dosyaya yazmadan önce, sadece bir metin değişkeni olarak görmek veya bir sunucuya göndermek istiyorsak dumps kullanırız.

```
import json

person_dict = {"name": "Ali", "languages": ["Python", "C#"]}

# Python sözlüğünü JSON metnine çeviriyoruz
# indent=4: JSON çıktısını girintili (okunabilir) yapar.
# sort_keys=True: Anahtarları alfabetik sıraya dizer.
result = json.dumps(person_dict, indent=4, sort_keys=True)

print(result)
print(type(result)) # <class 'str'> -> Artık bu bir metindir.
```

BTK AKADEMİ

Örnek Uygulama (json.dump)

- Elimizdeki verileri bilgisayara bir .json dosyası olarak kaydetmek için dump fonksiyonunu kullanırız.

```python
import json

person_dict = {
    "name": "Ebru Aydoğan",
    "languages": ["Python", "C#"],
    "isFullTime": True
}

# 'w' (write) modunda dosyayı açıyoruz
with open("person.json", "w", encoding="utf-8") as f:
    # Python sözlüğünü dosyaya yazıyoruz
    # ensure_ascii=False: Türkçe karakterlerin düzgün görünmesi için şart!
    json.dump(person_dict, f, indent=4, ensure_ascii=False)

print("Veriler person.json dosyasına başarıyla kaydedildi.")
```

BTK AKADEMİ

JSON Hiyerarşisi – Liste ve Sözlük Grupları

- Gerçek dünyada veriler nadiren tek bir anahtar-değer çiftinden oluşur. Genellikle bir anahtarın karşılığında bir **liste** (Array) veya başka bir **sözlük** (Object) bulunur.
- **JSON Listeleri:** Verilerin [ ] içinde tutulduğu yapılardır.
- **JSON Sözlük Listeleri:** Bir liste içindeki her bir elemanın ayrı birer sözlük olmasıdır. (Örn: Veritabanındaki kullanıcılar listesi).
- **Çoklu Gruplar:** İçiçe geçmiş, derin katmanlı yapılardır.

İçiçe JSON Yapılarına Erişim (Navigasyon)

- Bu karmaşık yapılarda istediğimiz veriye ulaşmak için "katmanları" tek tek geçmeliyiz. Önce liste indisini, sonra sözlük anahtarını kullanırız.

```python
import json

# İçiçe geçmiş (Nested) bir veri yapısı
data = {
    "okul_adi": "BTK Akademi",
    "siniflar": [
        {"ad": "Python-A", "mevcud": 20},
        {"ad": "Python-B", "mevcud": 25}
    ]
}

# 1. Katman: Okul adını al
print(data["okul_adi"]) # BTK Akademi

# 2. Katman: İlk sınıfın adını al
# data["siniflar"] -> Listeye ulaşır
# [0] -> Listenin ilk elemanına (sözlüğe) ulaşır
# ["ad"] -> O sözlükteki 'ad' anahtarına ulaşır
print(data["siniflar"][0]["ad"]) # Python-A
```

BTK AKADEMİ

JSON Verisini Güncelleme

- JSON dosyasındaki bir veriyi güncellemek için önce dosyayı load ile okumalı, Python üzerinde değişikliği yapmalı ve sonra dump ile tekrar üzerine yazmalıyız.

```python
import json

# 1. Önce oku
with open("data.json", "r") as f:
    db = json.load(f)

# 2. Python üzerinde listeye yeni eleman ekle
yeni_sinif = {"ad": "Python-C", "mevcud": 18}
db["siniflar"].append(yeni_sinif)

# 3. Güncel veriyi tekrar dosyaya yaz
with open("data.json", "w") as f:
    json.dump(db, f, indent=4)
```

BTK AKADEMİ

Sınıfları JSON'a Kaydetme (Serialization Class)

- **Büyük Problem:** Python'da kendi oluşturduğumuz bir sınıfın nesnesini (instance) doğrudan json.dump() ile kaydedemeyiz. Python, "Bu nesneyi nasıl metne çevireceğimi bilmiyorum" diyerek TypeError verir.
- **Neden?** Çünkü JSON sadece standart veri tiplerini (string, int, list, dict) tanır. Sizin özel sınıflarınızı tanımaz.
- **Çözüm:** Nesneyi kaydetmeden önce onu bir **Dictionary (Sözlük)** yapısına dönüştürmeliyiz.

Sınıfları JSON'a Kaydetme (Serialization Class)

- __dict__ Sihri ile Nesne Kaydetme
- Her Python nesnesinin içinde, o nesneye ait özellikleri (attribute) bir sözlük olarak tutan gizli bir __dict__ değişkeni bulunur. Bunu kullanarak nesneyi JSON'a "ikna" edebiliriz.

BTK AKADEMİ

Sınıfları JSON'a Kaydetme (Serialization Class)

```python
import json

class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

# Nesneyi oluşturuyoruz
p1 = Product("Samsung S23", 40000)

# Hatalı kullanım: json.dump(p1, f) -> HATA VERİR!

# Doğru kullanım: __dict__ ile sözlüğe çevirip kaydet
with open("products.json", "w") as f:
    # Nesnenin içindeki verileri sözlük olarak çıkart ve yaz
    json.dump(p1.__dict__, f, indent=4)

print("Nesne başarıyla JSON formatına çevrildi.")
```

BTK AKADEMİ

Uygulama 1 – Kargo Takip Sistemi Veri Hattı

- **Senaryo:** Bir kargo firmasının hatalı girilmiş verilerini düzeltip sisteme aktarmanız gerekiyor.
- **1. Veri Kaynağı (Ham Metin):**

```
raw_data = """
ID: 101 | Durum: TESLIM | Musteri: ali yılmaz | Tel: 05321112233
ID: 102 | Durum: IPTAL | Musteri: ayşe derya | Tel: 02164445566
ID: 103 | Durum: TESLIM | Musteri: mehmet can | Tel: 05447778899
"""
```

BTK AKADEMİ

Uygulama 1 – Kargo Takip Sistemi Veri Hattı

- 2. Beklenen Görevler:
  - **Adım 1 (Regex):** Ham metinden ID, Durum, Musteri Adı ve Telefon bilgilerini ayıklayın.
  - **Adım 2 (Manipülasyon):** Müşteri adlarının ilk harflerini büyüterek (Title Case) düzeltin.
  - **Adım 3 (CSV Yazma):** Tüm verileri kargo_listesi.csv dosyasına kaydedin.
  - **Adım 4 (JSON Filtreleme):** Sadece durumu IPTAL olan kargoları iptal_edilenler.json dosyasına, sözlük yapısında arşivleyin.

BTK AKADEMİ